

Documentação Técnica

Sistema Gestão Fácil

1. Visão Geral do Projeto

1.1. Nome do Projeto

Gestão Fácil

1.2. Objetivo Principal

O Gestão Fácil é um sistema de uso interno projetado para modernizar, centralizar e simplificar a gestão de contratos no âmbito da Justiça Federal (TRF-1 GO). O sistema substitui o método anterior, baseado em planilhas, por uma plataforma robusta, organizada e intuitiva.

1.3. O Problema Resolvido

Anteriormente, o gerenciamento de contratos era realizado através de planilhas (Excel), um método que, apesar de funcional, apresentava desafios significativos à medida que a complexidade e o volume de contratos aumentavam. Os principais problemas eram:

- **Falta de Centralização:** Múltiplas versões de arquivos circulavam entre os trabalhadores, gerando conflitos de dados, duplicação de conteúdo e incerteza sobre qual era a versão mais atualizada.
- **Controle de Prazos Manual:** A verificação de datas de vencimento de contratos era um processo manual e suscetível a falhas humanas, com alto risco de perda de prazos importantes.
- **Dificuldade de Rastreamento:** O registro e a consulta do histórico de alterações em um contrato (aditivos, reajustes) eram complexos e desorganizados.
- **Processo de Cadastro Ineficiente:** A criação de novos registros de contrato era repetitiva e pouco padronizada, aumentando a chance de erros.

1.4. Usuários-Alvo

Servidores e colaboradores do setor de contratos da Justiça Federal (TRF-1 GO) responsáveis pela criação, operação e fiscalização dos contratos da instituição.

2. Arquitetura do Sistema

2.1. Modelo Arquitetural

O **Gestão Fácil** é desenvolvido como uma **Aplicação Web Monolítica**.

Neste modelo, o framework Django é responsável tanto pela lógica do servidor quanto por renderizar as páginas HTML que são enviadas diretamente para o navegador do usuário (*Server-Side Rendering*).

2.2. Divisão Lógica

Apesar de ser um monólito, o sistema segue uma divisão lógica clara:

- **Aplicação (Backend + Frontend):** O código Python (Django) processa as requisições, aplica as regras de negócio e interage com o banco de dados. Ele também gera as páginas web dinamicamente usando templates HTML, CSS e JavaScript.
- **Banco de Dados (Data Layer):** O PostgreSQL atua como um serviço separado e independente, responsável exclusivamente pelo armazenamento e recuperação dos dados. A aplicação se comunica com ele para persistir todas as informações dos contratos.

2.3. Principais Tecnologias

- **Backend Framework:** Python (v3.10) com Django (v3.2.25).
- **Frontend:** HTML5, CSS3 e JavaScript (renderizados pelo sistema de templates do Django).
- **Banco de Dados:** PostgreSQL.

2.4. Comunicação Interna

A comunicação entre o "frontend" e o "backend" ocorre no seguinte fluxo:

1. O navegador do usuário faz uma requisição a uma URL do sistema.
2. O Django recebe essa requisição, executa a lógica de negócio correspondente (a *View*).
3. A *View* busca ou salva dados no banco de dados PostgreSQL através dos *Models*.
4. O Django renderiza um template HTML, injetando os dados necessários na página.
5. A página HTML final é enviada como resposta ao navegador.

3. Ambiente do Servidor e Procedimentos de Manutenção

Esta seção descreve a localização dos arquivos do projeto no servidor e detalha os comandos essenciais para a sua manutenção. O público-alvo são as equipes técnicas com acesso ao ambiente do servidor onde a aplicação está hospedada.

3.1. Localização do Projeto

Os arquivos da aplicação **Gestão Fácil** estão localizados no seguinte diretório do servidor:

```
/var/www/html/sistemas/gestaofacil-env
```

Este diretório será referido como **diretório raiz do projeto** ao longo desta documentação.

3.2. Arquivo de Configuração de Ambiente (**.env**)

O projeto utiliza um arquivo **.env** para gerenciar variáveis de ambiente, como credenciais de banco de dados e outras configurações sensíveis. Este arquivo **não faz parte do repositório Git** por razões de segurança e suas configurações prevalecem sobre as definições padrão no **settings.py**.

- **Localização:** O arquivo **.env** deve estar localizado no **diretório raiz do projeto**.
- **Propósito:** Centralizar todas as configurações que variam entre ambientes (desenvolvimento, produção) e guardar informações sigilosas.

[CONTINUAÇÃO DO TÓPICO 3.2 NA PÁGINA ABAIXO]

- **Estrutura do arquivo:**

```
DB_NAME='db_name'  
DB_USER='db_user'  
DB_PASSWORD='db_password'  
DB_HOST='db_host'  
DB_PORT='00000'  
  
SECRET_KEY='django_secret_key'  
DEBUG=True/False  
  
EMAIL_HOST_USER='example@email.com'  
EMAIL_HOST_PASSWORD='xxxx xxxx xxxx xxxx'  
  
TEAMS_URL='https://teams_post_url.api'  
  
WEBSITE_URL='https://srvintranet2-go.go.trf1.gov.br/'
```

3.3. Como Executar Comandos de Gerenciamento

Para executar qualquer comando de manutenção ou script do Django (arquivos `manage.py`), é **obrigatório** primeiro ativar o ambiente virtual (`venv`) do projeto.

1. **Acesse o diretório raiz do projeto** via terminal no servidor:

```
cd /var/www/html/sistemas/gestaofacil-env
```

2. **Ative o ambiente virtual:**

- a. No Windows:

```
Scripts\activate
```

- b. No Linux:

```
source gestaofacil-env/bin/activate
```

3. Após a ativação, o prompt do seu terminal será prefixado com `(gestaofacil-env)`, indicando que o ambiente está ativo. Todos os comandos `python` e `pip` a seguir usarão as versões e pacotes isolados deste ambiente.

3.4. Comandos de Manutenção Comuns

a) Aplicar Alterações no Banco de Dados (`migrate`)

- **Quando usar:** Após uma atualização de versão do sistema que inclua modificações na estrutura do banco de dados (nos `models` do Django).
- **Procedimento:**
 1. Ative o ambiente virtual (conforme passo 3.2).
 2. Execute o comando:

```
python manage.py migrate
```

b) Importar Contratos da Planilha (Procedimento de Contingência)

- **Quando usar:** Este é um procedimento especial, utilizado apenas em situações de contingência para uma carga ou restauração de dados a partir de uma planilha. **Não é uma operação de rotina.**
- **Procedimento:**
 1. Garanta que o arquivo a ser importado, nomeado exatamente como `Planilha_Contratos.xlsx`, esteja localizado na pasta `excel/` dentro do diretório raiz do projeto.
 2. Ative o ambiente virtual (conforme passo 3.2).
 3. Execute o comando:

```
python manage.py importar_contratos
```

4. Estrutura de Pastas e Arquivos

Esta seção detalha a organização dos principais diretórios e aplicações Django dentro do projeto, fornecendo um mapa para a equipe de manutenção localizar e entender os diferentes componentes do sistema.

4.1. Django Apps Principais

O projeto é modularizado em duas aplicações Django principais:

a) App **contratos**

- **Responsabilidade:** Este app é o núcleo da lógica de negócio e da manipulação de dados de contratos do sistema. Ele lida com tudo que é relacionado à entidade "Contrato" e suas operações no backend.
- **Componentes Chave:**
 - **Models (**models.py**):** Define a estrutura das tabelas do banco de dados para os contratos, fornecedores, aditivos, etc. É a fonte da verdade para os dados.
 - **Lógica de Negócio:** Contém os métodos e funções para as regras do sistema, como a verificação de status e a lógica para os alertas automáticos de vencimento.
 - **Serviços de Notificação:** Gerencia a integração e o envio de mensagens para serviços externos, como E-mail e Microsoft Teams.

b) App **pages**

- **Responsabilidade:** Este app é responsável por toda a renderização das páginas web, processamento de formulários e a lógica de apresentação que interage diretamente com o usuário.
- **Componentes Chave:**
 - **Views (**views.py**):** Contém a lógica para processar as requisições dos usuários, buscar os dados (usando o app **contratos**) e renderizar os templates HTML como resposta.
 - **Formulários (**forms.py**):** Define, valida e processa os formulários para criação e edição de contratos, garantindo a integridade dos dados inseridos pelo usuário.
 - **Lógica de Busca e Filtros:** Implementa a funcionalidade de pesquisa de contratos disponível na interface.

- **Templates Principais:**
 - `home.html`: Página principal do sistema, que funciona como um dashboard para visualizar, buscar e acessar os contratos para edição.
 - `contract.html`: Página utilizada para o registro de novos contratos e para a edição de contratos existentes.

4.2. Diretórios Importantes

a) `excel/`

- **Localização:** Na raiz do projeto.
- **Propósito:** Diretório de entrada para o script de importação de contingência. É aqui que o arquivo `Planilha_Contratos.xlsx` deve ser colocado para a execução do comando `importar_contratos`.

b) Gerenciamento de Arquivos Estáticos (`static`)

- **Observação Importante:** A gestão de arquivos estáticos neste projeto possui uma configuração centralizada no servidor. A pasta `static` **não reside dentro do diretório raiz do projeto**.
- **Localização:** Os arquivos estáticos (CSS, JavaScript, imagens) são servidos a partir de um diretório global, na seguinte pasta:

```
var/www/html/sistemas/static/gestaofacil
```

- **Manutenção:** Qualquer alteração em arquivos CSS, JS ou imagens deve ser realizada neste diretório centralizado. O comando `python manage.py collectstatic` é utilizado durante o deploy para coletar os arquivos estáticos de todos os apps do projeto e consolidá-los neste local.

5. Banco de Dados e Estrutura de Dados

O sistema utiliza PostgreSQL como banco de dados, e sua estrutura é definida e gerenciada através do ORM (Object-Relational Mapper) do Django. O design dos dados é centrado no model **Contrato**, com diversas outras tabelas relacionadas para armazenar informações complementares de forma organizada.

5.1. Principais Models (Tabelas)

A seguir, a descrição dos principais models definidos no app **contratos**:

- **Contrato**: A entidade central do sistema. Armazena todas as informações primárias de um contrato, como seu número, objeto, empresa contratada e status atual.
- **Vigencia**: Tabela com relação 1-para-1 com **Contrato**. É responsável por gerenciar todas as datas de vigência (original, atual e máxima), sendo crucial para os cálculos de vencimento.
- **ContratoHistorico**: Registra um log de todas as alterações feitas em um contrato, guardando quem fez a alteração e quando.
- **Gestor**: Armazena os dados do gestor ou do órgão gestor responsável pelo contrato, incluindo seu e-mail para notificações.
- **Contato**: Guarda os contatos (prepostos, fiscais) associados a um contrato. Um contrato pode ter múltiplos contatos.
- **TelefoneContato e EmailContato**: Tabelas auxiliares que armazenam os múltiplos telefones e e-mails que um **Contato** pode ter.
- **Garantia**: Contém os detalhes da garantia do contrato (se houver), como o tipo e o valor.
- **Links**: Tabela para armazenar URLs externas relevantes ao contrato, como links para planilhas e convenções no sistema SEI.
- **Notificacao**: Tabela de log que registra cada notificação de vencimento enviada, para qual contrato e em que data.
- **TokenGestorNotificacao e TokenContatoNotificacao**: Gerenciam tokens de validação únicos e com prazo de validade para as notificações, garantindo uma camada extra de controle e segurança.

5.2. Campos Essenciais para Manutenção

Para entender o funcionamento de um registro de contrato, os seguintes campos são de conhecimento fundamental:

1. **Contrato.numero_contrato:**

- **Descrição:** É o identificador único e legível do contrato no sistema (ex: "17/2024"). É gerado automaticamente pelo sistema para garantir a sequencialidade por ano.
- **Importância:** Chave de negócio principal para localizar e referenciar um contrato.

2. **Contrato.status:**

- **Descrição:** Indica o estado atual do ciclo de vida do contrato. Pode ser 'Ativo', 'Rescindido', 'Finalizado' ou 'Cancelado'.
- **Importância:** É o campo que determina se um contrato deve ser incluído em verificações de vencimento e outras operações de rotina. Contratos que não estão 'Ativo' são geralmente ignorados pelos alertas automáticos.

3. **Contrato.processo_sei:**

- **Descrição:** Armazena o número do processo do Sistema Eletrônico de Informações (SEI) associado ao contrato.
- **Importância:** Serve como principal ponto de referência para encontrar a documentação oficial do contrato fora do sistema Gestão Fácil.

4. **Vigencia.vigencia_atual:**

- **Descrição:** É a data de término da vigência corrente do contrato.
- **Importância:** **Este é o campo mais crítico para o sistema de alertas.** Todas as lógicas de "próximo ao vencimento" são calculadas com base na diferença entre esta data e a data atual.

5. **Contrato.proximo_ao_vencimento:**

- **Descrição:** Um campo **BooleanField** (Verdadeiro/Falso) que é atualizado por rotinas do sistema. Ele indica se o contrato entrou na "janela de alerta" (geralmente 180 dias antes da **vigencia_atual**).
- **Importância:** Funciona como um "gatilho" rápido para a interface e para os serviços de notificação, permitindo filtrar e destacar facilmente os contratos que exigem atenção imediata para renovação.

6. Módulos e Componentes Principais

Esta seção aprofunda o funcionamento de um dos módulos mais importantes do Gestão Fácil: o sistema de notificação automática sobre o vencimento de contratos.

6.1. Sistema de Notificação Automática de Vencimentos

a) Visão Geral

O sistema foi projetado para monitorar proativamente todos os contratos ativos e alertar as partes interessadas quando um contrato se aproxima da sua data de vencimento. O objetivo é evitar a perda de prazos para renovação ou encerramento, automatizando um processo que antes era manual e suscetível a falhas.

b) Gatilho e Execução (Cron Job)

A verificação é iniciada por um `cron job` configurado no servidor da aplicação.

- **Frequência:** Diariamente, à meia-noite (00:00).
 - **Comando:** O `cron job` dispara o seguinte comando de gerenciamento do Django:

```
python manage.py verificar_contratos
```

- **Exemplo de Configuração no Crontab:**

```
0 0 * * * /var/www/html/sistemas/gestaofacil-env/manage.py  
verificar_contratos
```

- *(Nota: Os caminhos devem ser ajustados de acordo com a configuração do servidor.)*

c) Fluxo Lógico

Quando o comando `verificar_contratos` é executado, ele identifica todos os contratos com `status 'ativo'` que estão dentro da janela de vencimento e, para cada um, inicia o processo de notificação.

d) Canais de Notificação

- **Microsoft Teams:** Aviso geral, de frequência **única**, enviado a um canal pré-configurado.
- **E-mail:** Aviso direcionado ao **Gestor** e **Contatos** do contrato, de frequência **recorrente** (a cada 7 dias), com um mecanismo de opt-out via link único.

e) Diagnóstico e Solução de Problemas

Esta seção serve como um guia para diagnosticar problemas comuns no sistema de notificação.

- **Nota Crítica sobre o Ambiente de Rede:** O servidor da aplicação opera dentro da rede privada da Justiça Federal (VPN), que possui **regras de firewall extremamente restritas**. Erros de falha de conexão ou timeouts ao tentar enviar notificações para serviços externos (servidor de e-mail, Microsoft Teams) são frequentemente causados por bloqueios de firewall.

Cenário 1: Nenhuma notificação (E-mail ou Teams) está sendo enviada.

- **Passos para Verificação:**
 1. **Verificar o Cron Job:** Confirme se o **cron job** está ativo (**crontab -l**) e se está sendo executado conforme os logs do sistema.
 2. **Executar o Comando Manualmente:** Ative o ambiente virtual e execute **python manage.py verificar_contratos** no terminal. Observe a saída para identificar qualquer erro de execução (traceback).
 3. **Verificar a Base de Dados:** Garanta que existam contratos com **status** 'ativo' e dentro da janela de vencimento para serem notificados.

Cenário 2: As notificações para um canal específico (ex: E-mail) não estão funcionando.

- **Passos para Verificação:**
 1. **Revisar Configurações de Ambiente (.env):** Verifique o arquivo **.env** na raiz do projeto.
 - **Para E-mails:** Confirme se as variáveis SMTP estão corretas. O e-mail configurado em **EMAIL_USER** deve ser o e-mail de serviço oficial fornecido e configurado pela equipe de TI da JF, pois qualquer outro pode ser bloqueado.
 - **Para Teams:** Confirme se a URL do webhook do Teams está correta.
 2. **Verificar o Firewall:** Esta é a causa mais provável para falhas de conexão. Se a execução manual do comando apresentar erros de

timeout ou falha na resolução de nomes (DNS), a equipe de manutenção deve contatar a equipe de infraestrutura de rede da JF para verificar se as regras de firewall permitem o tráfego de saída do servidor da aplicação para:

- O host e a porta do servidor de e-mail (SMTP).
- O domínio da API do Microsoft Teams.

3. **Consultar Logs da Aplicação:** Investigue os logs do Django para erros específicos, como `SMTPAuthenticationError` (credenciais de e-mail erradas) ou erros HTTP ao se comunicar com o Teams.

Cenário 3: Um contrato específico que deveria ser notificado não está recebendo alertas.

- **Passos para Verificação:**

1. **Inspecionar Dados do Contrato:** Verifique no banco de dados se o `status` do contrato é `'ativo'` e se a data em `vigencia_atual` está correta.
2. **Verificar Histórico de Notificações (Teams):** Consulte a tabela `Notificacao` e verifique se já existe um registro para esse contrato com `enviado_teams` como `True`. Em caso afirmativo, o comportamento está correto.
3. **Verificar Status de Opt-Out (E-mail):** Verifique se os destinatários (gestor ou contatos) já utilizaram o link para cancelar o recebimento dos alertas.